INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	:	Rabu, 4 Oktober 2026
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	09/ 02
Aktivitas	:	Implementasi — String Manipulation & Stream I/O dalam Java
Durasi	:	1 x 50 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Akhir Sesi
Tujuan	:	

1. Memahami dan menerapkan berbagai metode manipulasi string di Java
2. Menguasai konsep stream I/O untuk operasi file
3. Mampu mengintegrasikan string manipulation dengan file I/O
4. Membangun program yang robust dengan exception handling

BAGIAN 1: STRING MANIPULATION (20 Menit)
Langkah 1.1: Basic String Operations

Buat file: `StringManipulation.java`

```
import java.util.*;

public class StringManipulation {
    public static void main(String[] args) {
        String text = "  Hello World! Welcome to Java Programming  ";

        System.out.println("=== BASIC STRING OPERATIONS ===");
        System.out.println("Original: '" + text + "'");
        System.out.println("Length: " + text.length());
        System.out.println("Trimmed: '" + text.trim() + "'");
        System.out.println("Uppercase: '" + text.toUpperCase() + "'");
        System.out.println("Lowercase: '" + text.toLowerCase() + "'");

        // Substring dan indexing
        System.out.println("\n=== SUBSTRING & INDEXING ===");
        System.out.println("Character at index 5: '" + text.charAt(5) + "'");
        System.out.println("Substring (7-12): '" + text.substring(7, 12) + "'");
        System.out.println("Index of 'World': " + text.indexOf("World"));
        System.out.println("Last index of 'a': " + text.lastIndexOf("a"));

        // Replacement dan comparison
        System.out.println("\n=== REPLACEMENT & COMPARISON ===");
        System.out.println("Replace 'Java' with 'Python': '" +
                           text.replace("Java", "Python") + "'");
        System.out.println("Contains 'Programming': " + text.contains("Programming"));
        System.out.println("Starts with 'Hello': " + text.startsWith("Hello"));
        System.out.println("Ends with 'Programming': " + text.endsWith("Programming"));

        // Split dan join
        System.out.println("\n=== SPLIT & JOIN ===");
        String[] words = text.trim().split(" ");
        System.out.println("Number of words: " + words.length);
        System.out.print("Words: ");
        for (String word : words) {
            System.out.print("[" + word + "] ");
        }
        System.out.println();


        // StringBuilder untuk efisiensi
        System.out.println("\n=== STRING BUILDER ===");
        StringBuilder sb = new StringBuilder();
        sb.append("Java");
        sb.append(" Programming");
        sb.insert(0, "Learn ");
        sb.reverse();
        System.out.println("Reversed: " + sb.toString());
    }
}
```

Output:

```
=== BASIC STRING OPERATIONS ===
Original: 'Hello World! Welcome to Java Programming'
Length: 40
Trimmed: 'Hello World! Welcome to Java Programming'
Uppercase: 'HELLO WORLD! WELCOME TO JAVA PROGRAMMING'
Lowercase: 'hello world! welcome to java programming'

=== SUBSTRING & INDEXING ===
Character at index 5: ' '
Substring (7-12): 'orld!'
Index of 'World': 6
Last index of 'a': 34

=== REPLACEMENT & COMPARISON ===
Replace 'Java' with 'Python': 'Hello World! Welcome to Python Programming'
Contains 'Programming': true
Starts with 'Hello': true
Ends with 'Programming': true

=== SPLIT & JOIN ===
Number of words: 6
Words: [Hello] [World!] [Welcome] [to] [Java] [Programming]

=== STRING BUILDER ===
Reversed: gnimmargorP avaJ nraeL
```

Pertanyaan:

1. Mengapa `startsWith("Hello")` mengembalikan `false`?
2. Apa perbedaan antara `String` dan `StringBuilder`?
3. Bagaimana cara mengambil hanya kata "Java" dari string tersebut?

Modifikasi:

- Ubah kode untuk menampilkan hanya kata-kata yang panjangnya lebih dari 4 karakter
- Buat method untuk menghitung jumlah huruf vokal dalam string

BAGIAN 2: STREAM I/O DASAR (20 Menit)
Langkah 2.1: Membaca dan Menulis File

Buat file: `FileIOExample.java`

```java
import java.util.*;
import java.io.*;

public class FileIOExample {
    public static void main(String[] args) {
        String inputFile = "data.txt";
        String outputFile = "output.txt";

        // Tulis data ke file
        writeToFile(inputFile);

        // Baca data dari file
        readFromFile(inputFile);

        // Process dan tulis ke file baru
        processFile(inputFile, outputFile);
    }

    public static void writeToFile(String filename) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            writer.write("Alice,25,Computer Science");
            writer.newLine();
            writer.write("Bob,30,Mathematics");
            writer.newLine();
            writer.write("Charlie,22,Physics");
            writer.newLine();
            writer.write("Diana,28,Chemistry");
            writer.newLine();

            System.out.println("Data berhasil ditulis ke " + filename);
        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }

    public static void readFromFile(String filename) {
        System.out.println("\n=== MEMBACA FILE ===");
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            int lineNumber = 1;

            while ((line = reader.readLine()) != null) {
                System.out.println("Line " + lineNumber + ": " + line);
                lineNumber++;
            }

            System.out.println("File berhasil dibaca");
        } catch (FileNotFoundException e) {
            System.out.println("File tidak ditemukan: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
```

```java
    public static void processFile(String inputFile, String outputFile) {
        System.out.println("\n=== MEMPROSES FILE ===");
        try (BufferedReader reader = new BufferedReader(new FileReader(inputFile));
             BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile))) {

            String line;
            writer.write("NAME,AGE,MAJOR,STATUS");
            writer.newLine();

            while ((line = reader.readLine()) != null) {
                // Split data berdasarkan koma
                String[] data = line.split(",");
                String name = data[0].toUpperCase();
                int age = Integer.parseInt(data[1]);
                String major = data[2];
                String status = (age > 25) ? "GRADUATE" : "UNDERGRADUATE";

                // Tulis data yang telah diproses
                String processedLine = String.format("%s,%d,%s,%s", name, age, major, status);
                writer.write(processedLine);
                writer.newLine();

                System.out.println("Processed: " + processedLine);
            }

            System.out.println("File berhasil diproses dan disimpan sebagai " + outputFile);

        } catch (IOException e) {
            System.out.println("Error processing file: " + e.getMessage());
        }
    }
}
```

Output:

```
Data berhasil ditulis ke data.txt

=== MEMBACA FILE ===
Line 1: Alice,25,Computer Science
Line 2: Bob,30,Mathematics
Line 3: Charlie,22,Physics
Line 4: Diana,28,Chemistry
File berhasil dibaca

=== MEMPROSES FILE ===
Processed: ALICE,25,Computer Science,UNDERGRADUATE
Processed: BOB,30,Mathematics,GRADUATE
Processed: CHARLIE,22,Physics,UNDERGRADUATE
Processed: DIANA,28,Chemistry,GRADUATE
File berhasil diproses dan disimpan sebagai output.txt
Error writing file: data.txt (Permission denied)
```

Pertanyaan:

1. Apa keuntungan menggunakan `try-with-resources`?
2. Mengapa perlu menangani `FileNotFoundException` dan `IOException` secara terpisah?
3. Apa yang terjadi jika file data.txt sudah ada sebelum program dijalankan?

Modifikasi:

- Tambahkan exception handling untuk NumberFormatException
- Buat method untuk membaca file dan menghitung rata-rata usia
- Modifikasi program untuk menambahkan data baru ke file tanpa menghapus yang lama

BAGIAN 3: INTEGRASI & STUDI KASUS (20 Menit)
Langkah 3.1: Program Text Processing yang Komprehensif

Buat file: `TextProcessor.java`

```
import java.io.*;
import java.util.*;

public class TextProcessor {
    public static void main(String[] args) {
        String sourceFile = "source.txt";
        String processedFile = "processed.txt";
        String reportFile = "report.txt";

        // Buat file sumber contoh
        createSampleFile(sourceFile);

        // Process file: uppercase + statistics
        processTextFile(sourceFile, processedFile, reportFile);

        // Tampilkan report
        displayReport(reportFile);
    }

    public static void createSampleFile(String filename) {
        String[] sampleText = {
            "Java programming is fun and powerful.",
            "Streams and Files are important concepts.",
            "Exception handling makes code robust.",
            "String manipulation is frequently used.",
            "Practice makes perfect in programming."
        };

        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            for (String line : sampleText) {
                writer.write(line);
                writer.newLine();
            }
            System.out.println("Sample file created: " + filename);
        } catch (IOException e) {
            System.out.println("Error creating sample file: " + e.getMessage());
        }
    }

    public static void processTextFile(String inputFile, String outputFile, String reportFile) {
        try (BufferedReader reader = new BufferedReader(new FileReader(inputFile));
             BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile));
             BufferedWriter reportWriter = new BufferedWriter(new FileWriter(reportFile))) {

            String line;
            int totalLines = 0;
            int totalWords = 0;
            int totalChars = 0;
            Map<Character, Integer> charFrequency = new HashMap<>();
```

```
            while ((line = reader.readLine()) != null) {
                totalLines++;

                // Process line: convert to uppercase
                String processedLine = line.toUpperCase();
                writer.write(processedLine);
                writer.newLine();

                // Calculate statistics
                String[] words = line.split("\\s+");
                totalWords += words.length;
                totalChars += line.length();

                // Character frequency
                for (char c : line.toLowerCase().toCharArray()) {
                    if (Character.isLetter(c)) {
                        charFrequency.put(c, charFrequency.getOrDefault(c, 0) + 1);
                    }
                }
            }

            // Write report
            reportWriter.write("=== TEXT PROCESSING REPORT ===");
            reportWriter.newLine();
            reportWriter.write("Total Lines: " + totalLines);
            reportWriter.newLine();
            reportWriter.write("Total Words: " + totalWords);
            reportWriter.newLine();
            reportWriter.write("Total Characters: " + totalChars);
            reportWriter.newLine();
            reportWriter.write("Average Words per Line: " + (double) totalWords / totalLines);
            reportWriter.newLine();
            reportWriter.newLine();
            reportWriter.write("CHARACTER FREQUENCY:");
            reportWriter.newLine();

            // Sort characters by frequency
            List<Map.Entry<Character, Integer>> sortedChars = new ArrayList<>(charFrequency.entrySet());
            sortedChars.sort((a, b) -> b.getValue().compareTo(a.getValue()));

            for (Map.Entry<Character, Integer> entry : sortedChars) {
                reportWriter.write(entry.getKey() + ": " + entry.getValue());
                reportWriter.newLine();
            }

            System.out.println("File processing completed!");
            System.out.println("   - Input: " + inputFile);
            System.out.println("   - Output: " + outputFile);
            System.out.println("   - Report: " + reportFile);

        } catch (IOException e) {
            System.out.println("Error processing files: " + e.getMessage());
        }
    }

    public static void displayReport(String reportFile) {
        System.out.println("\n=== PROCESSING REPORT ===");
        try (BufferedReader reader = new BufferedReader(new FileReader(reportFile))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading report: " + e.getMessage());
        }
    }
}
```

Output:

```
Sample file created: source.txt
File processing completed!
- Input: source.txt
- Output: processed.txt
- Report: report.txt

=== PROCESSING REPORT ===
=== TEXT PROCESSING REPORT ===
Total Lines: 5
Total Words: 30
Total Characters: 182
Average Words per Line: 6.0

CHARACTER FREQUENCY:
e: 18
a: 16
i: 14
n: 14
...
```

Pertanyaan:

1. Bagaimana cara kerja HashMap untuk menghitung frekuensi karakter?
2. Mengapa kita menggunakan `\\s+` untuk split words daripada spasi biasa?
3. Apa keuntungan menggunakan `BufferedReader/BufferedWriter`?

Tugas Modifikasi:

1. Mudah: Modifikasi program untuk menghitung jumlah kalimat (berdasarkan tanda titik).
2. Sedang: Tambahkan fitur untuk menemukan kata yang paling sering muncul.
3. Sulit: Buat program yang bisa memproses file CSV dan menghasilkan summary statistics.

PENGUMPULAN:

1. Kumpulkan semua hasil pekerjaan Anda dalam folder ZIP dengan penamaan filename: **PBO_W9S2_NIM.zip**
2. Anda wajib menambahkan penjelasan berupa komen pada baris/blok di code Anda.

Selamat Mengerjakan ! ☺